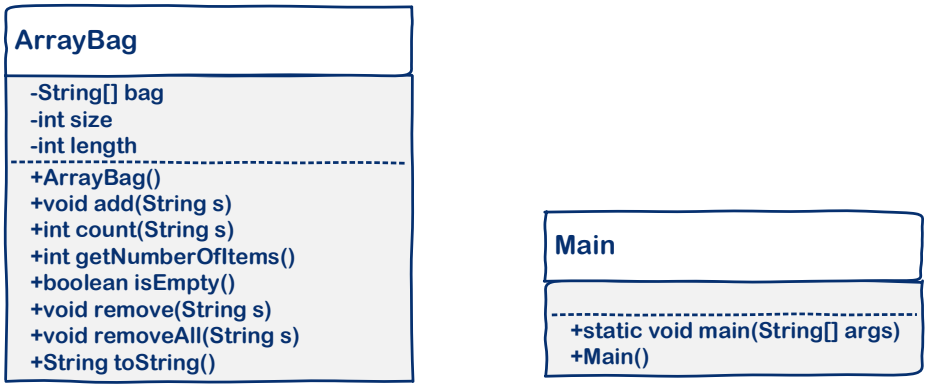
COP3530 – Assignment 2

Objective

Students will practice the use of arrays and abstract data types and will be able to develop a solution to a problem involving implementation of a bag abstract data type.

Assignment Problem

The Bag ADT is an abstraction of a multiset, i.e. a set where an item can be present in the bag more than once. Locations of items are not relevant in a bag, so operations such as `removeItemAt(int location)` or `retrieveItemAt(int location)` are not allowed. Implement a Bag ADT using an array, as described in the UML class diagram and the instructions below.



Guidelines

- The assignment is to be completed individually or in teams of two students. Questions are based on the content studied in class on time complexity, list ADT, Bag ADT, and arrays.

- You are allowed to use all of the code given in the lectures. In those cases, make sure you properly credit its source.

- Classes from the Java Collection Framework, e.g. java.util.ArrayList, are not allowed for use in your implementation.

- Students are required to structure the code as indicated in the UML class diagrams.

- Several files accompany this project description:
    - *assignment 2 test set.txt*: a test file,
    - *Main.java*: a complete implementation of the `Main` class (tests your code using file *assignment 2 test set.txt*),
    - *ArrayBag.java*: the framework of the `ArrayBag` class, with the constructor and the `toString` method implemented,
    - *output.pdf*: a file with the output generated by the `Main` class provided.

- Students are required to use the given class frameworks included, completing all the methods as indicated in the Javadoc comment section of each method.

- The toString method provided must be used in all output. Students are not allowed to modify it or use other form of output.

- The output of your program is required to be the same as the output example provided.

Deliverables

• A compressed folder, *PID Assignment 2 (for example, 1234567 Assignment 2)*, containing the files:

- *ArrayBag.java*,
- A screenshot of the running program. Note: a screenshot is an image that shows 1) the IDE environment with code and 2) the output window. More than one image file might be submitted.

• Include **only** the .java files mentioned above; do not include other files or folders generated by the IDE.

• Project submission containing .class files instead of .java files will be considered unsubmitted and will be given a grade of 0.

• Make sure you write the full name, class section, and Panther ID of each author in the @author field of the Java classes.

• Team submission: each team member must enter all names, class sections, and PIDs of the team in the comment window of the project dropbox. Select a team leader to submit the project (only team leaders submit the project, but the info of the team members must be entered by each student in the team).

Grading Rubric

The assignment is worth 100 points (out of 1000 total course points). Grade components:

<table>
<tr><th>Component</th><th>Points</th><th>Description</th></tr>
<tr><td>Submission</td><td>3</td><td>The student has submitted the project solution using the requirements for deliverables specified in the Deliverables section.</td></tr>
<tr><td>Organization</td><td>3</td><td>Code is expected to be neat, organized, and readable.</td></tr>
<tr><td>Content</td><td>94</td><td><table><tr><th>Element</th><th>Points</th></tr><tr><td>ArrayBag.java</td><td>90 pts</td></tr><tr><td>Screenshot(s)</td><td>4</td></tr></table></td></tr>
</table>